# Axolotl

1. What is Axolotl
2. Why config-driven matters
3. What Axolotl can do that HF alone cannot
4. Training methods
5. Dataset & scalability
6. Difference between core hf, unsloth, llamafactory, axolotl
7. Live Collab and Docker demo (Finetuning with minimum steps)

## 1. What is Axolotl?

Axolotl = config-driven (CLI-first) framework with Python extensibility

Axolotl supports two ways of running training:
one via the CLI using YAML config files,
and another programmatic approach using Python config classes

Open source
Free to use
Industry-oriented
built on Hugging Face ecosystem

## 2. Axolotl = CONFIG-DRIVEN TRAINING ENGINE

This is Axolotl's biggest strength
One YAML controls:

Dataset loading
Tokenization
Quantization
Training
Evaluation
Inference

"Unlike Hugging Face where code controls training, Axolotl flips it: YAML controls everything."

### Why config-driven matters

Reproducibility
Faster Experiment (Research + Production both)
Automation & CI/CD Friendly

## 3. What Axolotl can do that HF alone cannot

Features:

- **Multiple Model Support**: Train various models like GPT-OSS, LLaMA, Mistral, Mixtral, Pythia, and many more models available on the Hugging Face Hub.
- **Multimodal Training**: Fine-tune vision-language models (VLMs) including LLaMA-Vision, Qwen2-VL, Pixtral, LLaVA, SmolVLM2, and audio models like Voxtral with image, video, and audio support.
- **Training Methods**: Full fine-tuning, LoRA, QLoRA, GPTQ, QAT, Preference Tuning (DPO, IPO, KTO, ORPO), RL (GRPO), and Reward Modelling (RM) / Process Reward Modelling (PRM).
- **Easy Configuration**: Re-use a single YAML configuration file across the full fine-tuning pipeline: dataset preprocessing, training, evaluation, quantization, and inference.
- **Performance Optimizations**: Multipacking, Flash Attention, Xformers, Flex Attention, Liger Kernel, Cut Cross Entropy, Sequence Parallelism (SP), LoRA optimizations, Multi-GPU training (FSDP1, FSDP2, DeepSpeed), Multi-node training (Torchrun, Ray), and many more!
- **Flexible Dataset Handling**: Load from local, HuggingFace, and cloud (S3, Azure, GCP, OCI) datasets.
- **Cloud Ready**: We ship Docker images and also PyPI packages for use on cloud platforms and local hardware.

| Feature | Hugging Face | Axolotl |
|---|---|---|
| Multiple Model Support | ✓ | ✓ |
| Multimodal Training | ✓ | ✓ |
| Training Methods | ✓ | ✓ |
| Easy Configuration (Single YAML) | ✗ | ✓ |
| Flexible Dataset Handling | ✓ | ✓ |
| Cloud Ready (Docker + PyPI) | ✓ | ✓ |

## 4. Training Methods Supported

"Axolotl is not just LoRA — it's a full post-training platform."

Fine-tuning:
Full FT
LoRA / QLoRA
GPTQ
QAT

Preference / RL:
DPO, IPO, KTO, ORPO
GRPO
RM, PRM

## 5. Dataset Support

Local
Hugging Face
Cloud (S3, Azure, GCP, OCI)
Instruction, chat, preference, vision, audio

## 6. Diff between HF/Unsloth/LLaMA-Factory/Axolotl

| # | Dimension | Hugging Face (HF) | Unsloth | LLaMA-Factory | Axolotl |
|---|---|---|---|---|---|
| 1 | Core Purpose | Base ML framework for model training & inference | Performance engine that accelerates HF | End-to-end fine-tuning platform with UI | Training orchestrator for scalable experiments |
| 2 | Performance (Speed & VRAM) | Baseline speed, high VRAM usage | 2–3× faster, 50–80% less VRAM (verified) | Same as HF (no kernel optimization) | Same as HF (no kernel optimization) |
| 3 | GPU-level Optimizations | None (uses standard PyTorch ops) | Custom CUDA + Triton kernels, manual backprop | None | None |
| 4 | Workflow & Scale | Code-heavy, manual setup | Simple code, single-GPU focused | UI + CLI driven, structured but less scalable | YAML-driven, reproducible, multi-GPU, DeepSpeed/FSDP |

## Performance Optimization – One-Line, Strong Definitions

**Multipacking**
Packs multiple short sequences into a single training batch to maximize GPU utilization and eliminate padding waste.

Flash Attention
A memory-efficient attention algorithm that computes attention without materializing full matrices, enabling faster training and longer contexts.

xFormers
A modular library providing optimized transformer building blocks, including efficient attention kernels and memory-aware ops.

Flex Attention
A flexible attention framework that supports custom sparsity patterns and dynamic attention layouts for performance and experimentation.

Liger Kernel
A fused GPU kernel suite that combines multiple transformer operations to reduce memory bandwidth and kernel launch overhead.

Cut Cross Entropy
An optimized loss computation that avoids unnecessary token processing, reducing memory usage and speeding up backpropagation.

Sequence Parallelism (SP)
Splits long input sequences across GPUs instead of model weights, enabling scalable long-context training.

LoRA Optimizations
Training efficiency techniques that restrict updates to low-rank adapter matrices, dramatically reducing compute and memory costs.

## Distributed & Large-Scale Training

FSDP (Fully Sharded Data Parallel) – Stage 1
Shards model parameters across GPUs to reduce memory footprint while keeping optimizer states replicated.

FSDP (Fully Sharded Data Parallel) – Stage 2
Shards both model parameters and optimizer states, enabling training of much larger models on limited GPU memory.

DeepSpeed
A distributed training framework that combines optimizer sharding, memory offloading, and pipeline parallelism for massive models.

Multi-GPU Training
Parallelizes training across multiple GPUs to increase throughput and handle larger models or batches.

Torchrun
PyTorch’s native launcher for distributed training that manages process groups and GPU communication.

Ray
A distributed execution framework that orchestrates multi-node, multi-GPU workloads with fault tolerance and dynamic scaling.

| Optimization | Hugging Face Alone | Axolotl |
|---|---|---|
| **Multipacking** | ❌ Manual custom collator | ✅ Built-in |
| **Flash Attention** | ⚠️ Manual enable + version hell | ✅ Auto-handled |
| **xFormers** | ⚠️ Manual install + config | ✅ Plug & play |
| **Flex Attention** | ❌ Research-level | ⚠️ Limited / experimental |
| **Liger Kernel** | ❌ Not native | ✅ Integrated |
| **Cut Cross Entropy** | ❌ Custom loss | ✅ Built-in |
| **Sequence Parallelism (SP)** | ❌ Very complex | ⚠️ Partial / advanced |
| **LoRA optimizations** | ⚠️ PEFT tuning needed | ✅ Optimized defaults |
| **Multi-GPU (FSDP1/FSDP2)** | ⚠️ Manual painful | ✅ One-flag |
| **DeepSpeed** | ⚠️ JSON + code glue | ✅ Stable configs |
| **Multi-node training** | ⚠️ torchrun setup | ✅ torchrun / Ray ready |